



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 09

NOMBRE COMPLETO: Velazquez Caudillo Osbaldo

N° de Cuenta: 320341704

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 06

SEMESTRE 2026-1

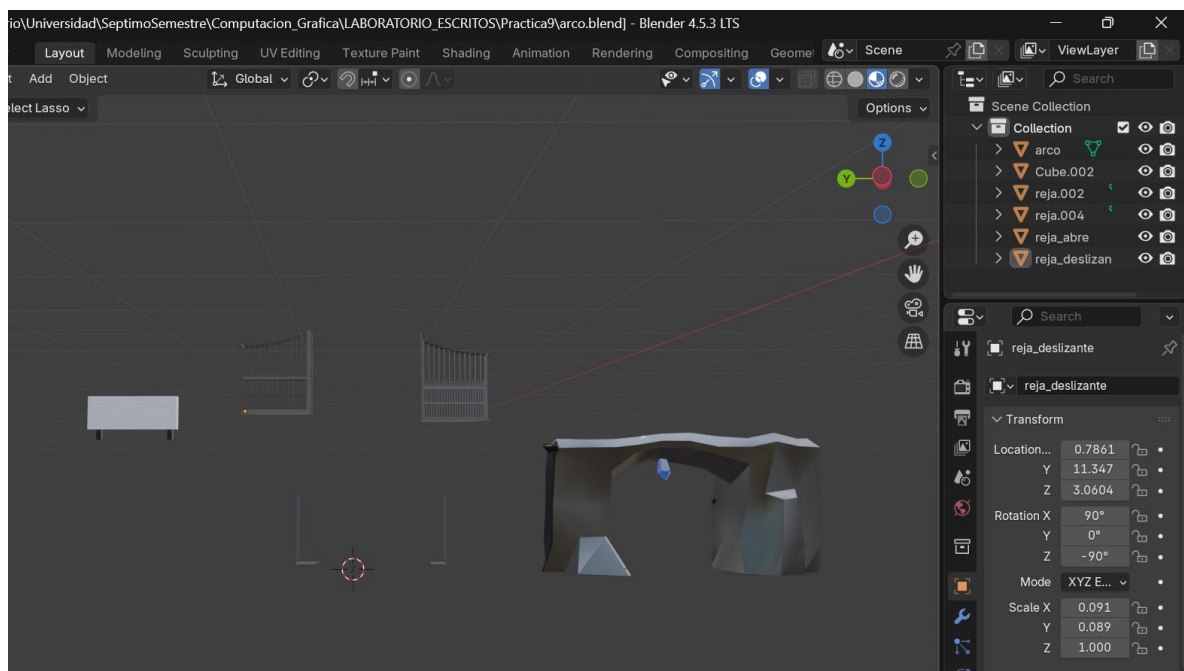
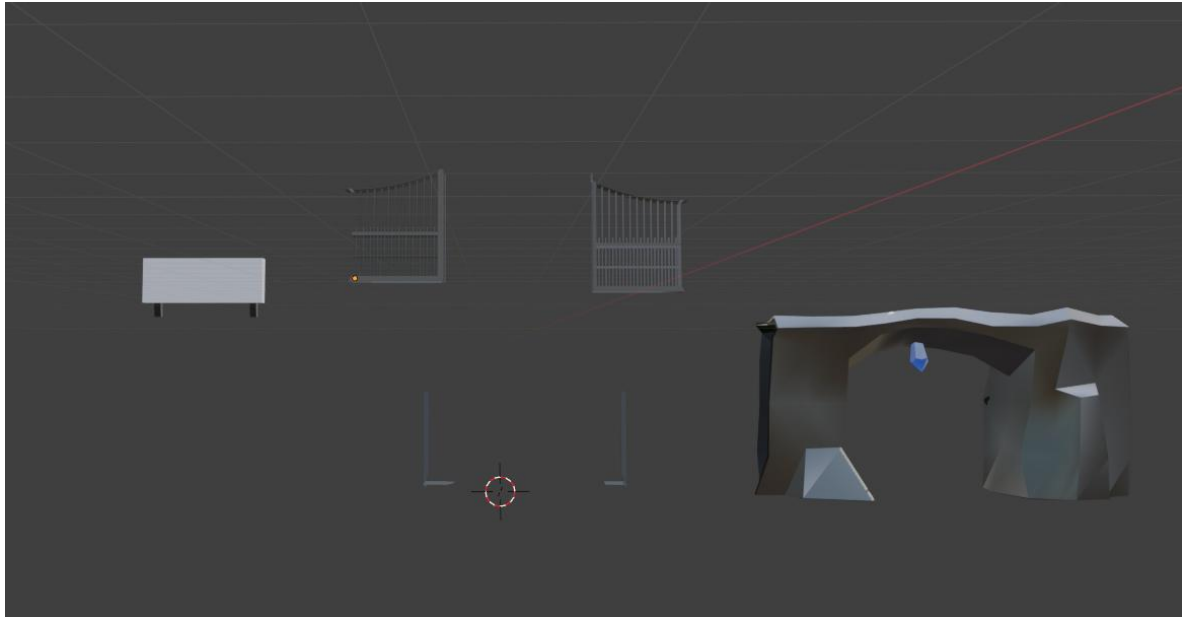
FECHA DE ENTREGA LÍMITE: 02/11/25

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1.- Separar del arco la parte del letrero y las puertas o rejas



En Blender separamos tanto el arco, así como la reja en 4 partes que en si las 2 partes que corresponden al marco por así decirlo que es el que va pegado al arco

para que sostenga a las rejas, y por ultimo el letrero que ese únicamente lo texturizamos de las patas que tiene para que vaya unido al arco.

Después las importamos para trabajar en ellas por medio de código en OPENGL.

Creamos las variables de tipo Model que corresponden a las partes de la reja, el arco y el cartel.

```
Model Arco_M;  
Model Reja_Deslizante;  
Model Reja_Abre;  
Model Cartel_Reja;
```

Le asignamos el modelo a las variables.

```
Arco_M.LoadModel("Models/arco.obj");  
Reja_Deslizante = Model();  
Reja_Deslizante.LoadModel("Models/reja_deslizante.obj");  
Reja_Abre = Model();  
Reja_Abre.LoadModel("Models/reja_abre.obj");  
Cartel_Reja = Model();  
Cartel_Reja.LoadModel("Models/cartel_reja.obj");
```

Y ya únicamente unimos cada parte por jerarquía.

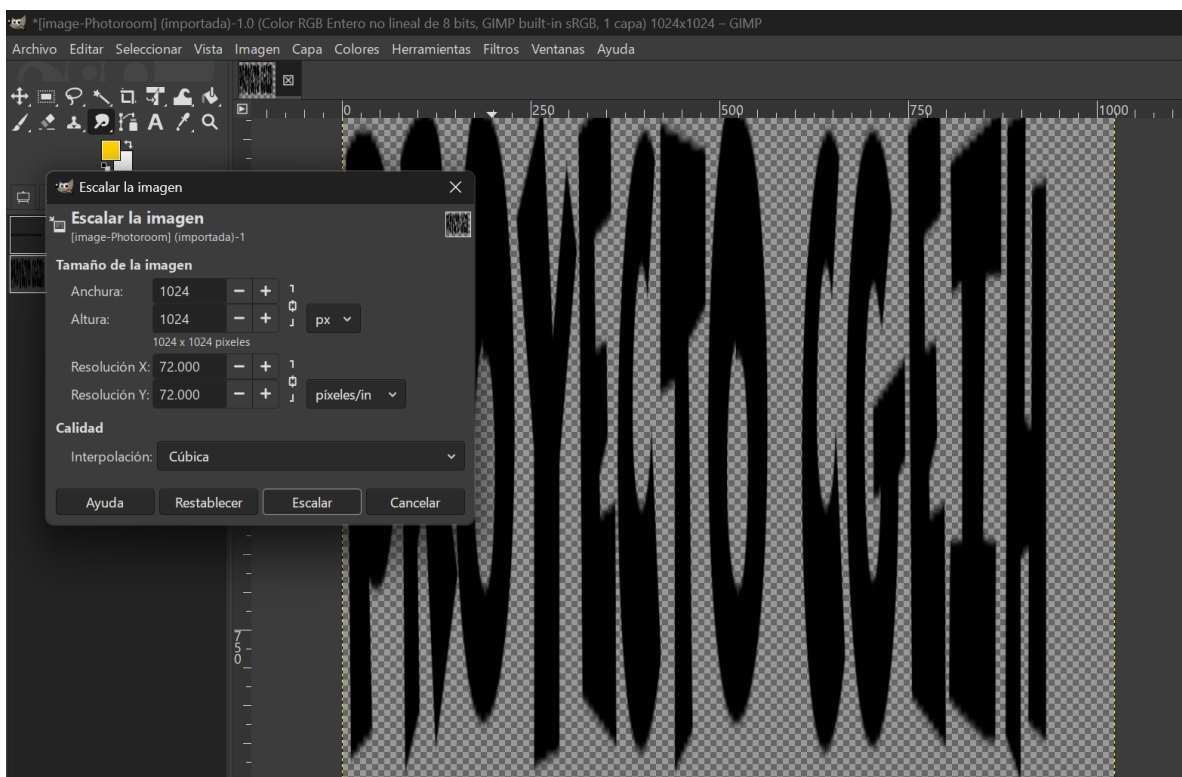
```
//Instancia del arco  
model = glm::mat4(1.0);  
model = glm::translate(model, glm::vec3(4.0f, 7.7f, -2.0f));  
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));  
modelaux = model;  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
Arco_M.RenderModel();  
  
//Reja que se abre  
model = modelaux;  
model = glm::translate(model, glm::vec3(-3.1199f, -6.48f, -5.5469f));  
model = glm::rotate(model, -puertas_rejas * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
Reja_Abre.RenderModel();  
  
//Reja que se desliza  
model = modelaux;  
model = glm::translate(model, glm::vec3(-2.7708f, -9.341, 5.8329f + (puertas_rejas / 18)));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
Reja_Deslizante.RenderModel();  
  
//Cartel  
model = modelaux;  
model = glm::translate(model, glm::vec3(-0.08328f, 2.642f, -0.17226f));  
modelaux = model;  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
Cartel_Reja.RenderModel();
```

Ejecución



2.-Hacer que en el arco que crearon se muestre la palabra: PROYECTO CGEIHG desplazándose las letras de derecha a izquierda como si fuera letrero LCD/LED de forma cíclica

Para esto lo que hacemos es crear la palabra tal cual que queremos sin fondo, después en GIMP nada más la optimizamos para que sea cuadrada potencia de 2



Ya teniendo la imagen la exportamos y la agregamos a la carpeta Textures.

Despues creamos la variable en el código

```
Texture CartelReja;
```

Asignamos la textura

```
CartelReja = Texture("Textures/cartel_tipografia.png");  
CartelReja.LoadTextureA();
```

Despues para que el texto quede en donde esta el cartel lo asignamos por jerarquía dentro del código.

Tomamos la traslación.

```
//Cartel  
model = modelaux;  
model = glm::translate(model, glm::vec3(-0.08328f, 2.642f, -0.17226f));  
modelaux= model;  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
Cartel_Reja.RenderModel();
```

Y en la textura lo asignamos para que únicamente nos quede acomodarlo directamente en lo blanco del cartel.

```
model = modelaux;  
model = glm::translate(model, glm::vec3(0.4f, 0.0f, 0.25f)); //  
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));  
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));  
model = glm::scale(model, glm::vec3(6.8f, 1.0f, 2.2f)); //horizontal x, vertical z  
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
CartelReja.UseTexture();  
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);  
meshList[7]->RenderMesh();  
glDisable(GL_BLEND);
```

Ahora para el movimiento de las letras fue como el ejercicio de laboratorio

```
//Letras en movimiento
toffsetnumerocambiau += 0.002 * deltaTime;
if (toffsetnumerocambiau > 1.0)
    toffsetnumerocambiau = 0.0;
toffsetnumerov = 0.0;
toffset = glm::vec2(toffsetnumerocambiau, toffsetnumerov);
model = modelaux;
```

Este código hace que la textura de mis letras se desplace en U (horizontal). Incremento toffsetnumerocambiau con $0.002 * \text{deltaTime}$ para que se mueva suave, si llega a 1.0, lo reinicio a 0.0 para repetir el ciclo.

Si ese valor se pasa de 1.0 lo reinicio a 0.0 para que la animación se repita en bucle.

Al final armo el vector toffset con `glm::vec2(toffsetnumerocambiau, toffsetnumerov);` para que la textura parezca desplazarse horizontalmente.

Ejecución



3.- Hacer que al presionar una tecla una de las puertas se abra por medio de una rotación y la otra puerta se abra por medio de un deslizamiento.

Para lograr esta animación, una vez que ya tenemos los cuerpos acomodados en su lugar y por jerarquía tal y como se hizo en el ejercicio 1.

En el Window.h creamos abrirreja que nos ayuda a saber si si esta abriendo o no la reja por eso se creó de tipo bool.

```
bool abrirreja() { return rejaabierta; }
```

En el Window.cpp implementamos que si la tecla R esta presionada entonces será true, en caso de que la tecla se deje de presionar toma el valor de falso.

```
//REJAS
if (key == GLFW_KEY_R) {
    theWindow->rejaabierta = true;
}
if (key == GLFW_KEY_R && action == GLFW_RELEASE) {
    theWindow->rejaabierta= false;
}
```

En el archivo principal creamos la variable puertas rejas que nos ayudara a abrir y cerrar la reja.

```
float puertas_rejas;

// Movimiento lateral cuando
```

Inicializamos la variable

```
puertas_rejas = 0.0f;
```

Después lo que hacemos si abrir reja es true entonces puertas rejas se va a empezar a incrementar en 1 multiplicado por deltaTime pero esto únicamente si

puertas rejas es $\leq$ a 90 si es falso el valor se va a decrementar así es como funciona el movimiento.

```
if (mainWindow.abrirreja()) {  
    if (puertas_rejas <= 90) {  
        puertas_rejas += 1.0 * deltaTime;  
    }  
    else {  
        if (puertas_rejas > 0) {  
            puertas_rejas -= 1.0 * deltaTime;  
        }  
    }  
}
```

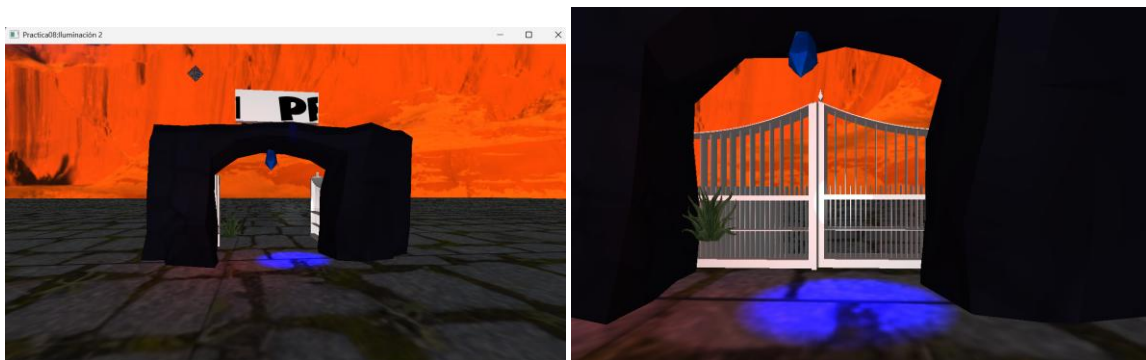
Para la reja que se abra de manera normal, lo que hacemos es mandarlo el valor de puertas rejas en un rotate, así de esta manera se va a ir moviendo sobre el eje y.

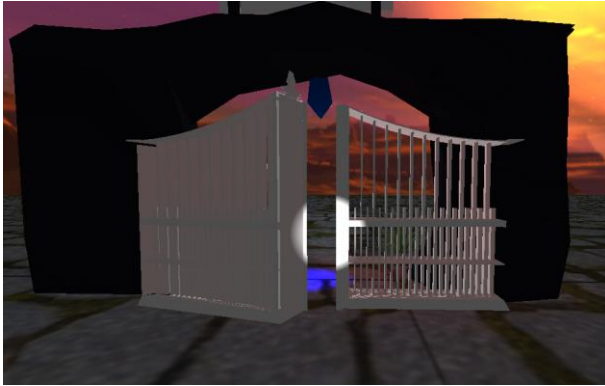
```
//Reja que se abre  
model = modelaux;  
model = glm::translate(model, glm::vec3(-3.1199f, -6.48f, -5.5469f));  
model = glm::rotate(model, -puertas_rejas * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
Reja_Abre.RenderModel();
```

Para obtener el movimiento deslizante solamente le sumamos el valor dividido entre 18 (este valor se eligió de manera tentativa) sobre el eje z para que se deslice desde la perspectiva que tenemos.

```
//Reja que se desliza  
model = modelaux;  
model = glm::translate(model, glm::vec3(-2.7708f, -9.341, 5.8329f + (puertas_rejas / 18)));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
Reja_Deslizante.RenderModel();
```

Ejecución





Extra 5 puntos: Su dado de 8 caras cae girando sobre el piso, y muestra un número random siempre hacia arriba, se repite la tirada al presionar una tecla.

```
// Movimiento lateral cuando rueda
float dadoLadoX;
float dadoLadoZ;

// Rotación del dado
float dadoRotX;
float dadoRotY;
float dadoRotZ;
```

Ponemos vértices del octaedro

```
void CrearDado()
{
    unsigned int cube_indices[] = {
        // cara 0
        0, 1, 2,
        // cara 1
        3, 4, 5,
        // cara 2
        6, 7, 8,
        // cara 3
        9, 10, 11,
        // cara 4
        12, 13, 14,
        // cara 5
        15, 16, 17,
        // cara 6
        18, 19, 20,
        // cara 7
        21, 22, 23
    };

    // average normals
    GLfloat cube_vertices[] = {
        // face 0 (top, front, right) 7 triángulos azules
        // 0
        0.0f, 0.5f, 0.0f, 0.25f, 0.50f, 0.577350f, 0.577350f, 0.577350f, //0
        0.0f, 0.0f, 0.5f, 0.51f, 0.75f, 0.577350f, 0.577350f, 0.577350f, //1
        0.5f, 0.0f, 0.0f, 0.51f, 0.75f, 0.577350f, 0.577350f, 0.577350f, //2

        // face 1 (top, right, back) 2 triángulos rojos
        0.0f, 0.5f, 0.0f, 0.75f, 0.50f, 0.577350f, 0.577350f, -0.577350f, //3
        0.5f, 0.0f, 0.0f, 0.49f, 0.25f, 0.577350f, 0.577350f, -0.577350f, //4
        0.0f, 0.0f, -0.5f, 0.99f, 0.25f, 0.577350f, 0.577350f, -0.577350f, //5

        // face 2 (top, back, left) 3 triángulos verdes
        0.0f, 0.5f, 0.0f, 0.75f, 0.50f, -0.577350f, 0.577350f, -0.577350f, //6
        0.0f, 0.0f, -0.5f, 0.49f, 0.25f, -0.577350f, 0.577350f, -0.577350f, //7
        -0.5f, 0.0f, 0.0f, 0.99f, 0.25f, -0.577350f, 0.577350f, -0.577350f, //8

        // face 3 (top, left, front) 6 triángulos verdes
        0.0f, 0.5f, 0.0f, 0.50f, 0.25f, -0.577350f, 0.577350f, 0.577350f, //9
        -0.5f, 0.0f, 0.0f, 0.75f, 0.50f, -0.577350f, 0.577350f, 0.577350f, //10
        0.0f, 0.0f, 0.5f, 0.25f, 0.50f, -0.577350f, 0.577350f, 0.577350f, //11

        // face 4 (bottom, right, front) 4 triángulos azules
        0.0f, -0.5f, 0.0f, 0.75f, 0.50f, 0.577350f, -0.577350f, 0.577350f, //12
        0.5f, 0.0f, 0.0f, 0.49f, 0.25f, 0.577350f, -0.577350f, 0.577350f, //13
        0.0f, 0.0f, 0.5f, 0.49f, 0.25f, 0.577350f, -0.577350f, 0.577350f, //14

        // face 5 (bottom, back, right) 5 triángulos verdes-rojos
        0.0f, -0.5f, 0.0f, 0.25f, 0.51f, 0.577350f, -0.577350f, -0.577350f, //15
        0.0f, 0.0f, -0.5f, 0.51f, 0.25f, 0.577350f, -0.577350f, -0.577350f, //16
        0.5f, 0.0f, 0.0f, 0.51f, 0.25f, 0.577350f, -0.577350f, -0.577350f, //17

        // face 6 (bottom, left, back) 9 triángulos rojos
        0.0f, -0.5f, 0.0f, 0.25f, 0.51f, -0.577350f, -0.577350f, -0.577350f, //18
        -0.5f, 0.0f, 0.0f, 0.50f, 0.25f, -0.577350f, -0.577350f, -0.577350f, //19
        0.0f, 0.0f, -0.5f, 0.51f, 0.25f, -0.577350f, -0.577350f, -0.577350f, //20

        // face 7 (bottom, front, left)
        0.0f, -0.5f, 0.0f, 0.50f, 0.75f, -0.577350f, -0.577350f, 0.577350f, //21
        0.0f, 0.0f, -0.5f, 0.25f, 0.50f, -0.577350f, -0.577350f, 0.577350f, //22
        -0.5f, 0.0f, 0.0f, 0.75f, 0.50f, -0.577350f, -0.577350f, 0.577350f, //23
    };

    Mesh* dado = new Mesh();
    dado->CreateMesh(cube_vertices, cube_indices, 192, 24);
    mesh.in.push_back(dado);
}
```

Textura del dado

```
datoTexture = Texture("Textures/dado_ocho_caras_optimizada.png");
datoTexture.LoadTextureA();
```

Window.h

```
bool dadocae() { return dadoy; }
```

Window.cpp

```
//DADO
if (key == GLFW_KEY_T && action==GLFW_PRESS) {
    theWindow->dadoy = !theWindow->dadoy;
}
```

Si se presiona la tecla T usando la función mainWindow.dadocae() se vuelve true lo que nos indica que debe de tener un movimiento hacia -y.

```
if (mainWindow.dadocae()) {
    if (dragonavance == 0.0f) {
        numeroSalio = (rand() % 8) + 1;
        //printf("Numero salio random:\n %f", numeroSalio);
        angulo = (rand() % 360) * toRadians;
        datoLadoX = cos(angulo) * 3.0f;
        datoLadoZ = sin(angulo) * 3.0f;
    }
    if (dragonavance > -12.0f) {
        dragonavance -= avanceoffset * deltaTime;
        // Girar mientras cae
        datoRotX += 300.0f * deltaTime;
        datoRotY += 250.0f * deltaTime;
        datoRotZ += 200.0f * deltaTime;
    }
}
```

Cuando es la primera vez que se presiona T, detecto esto verificando si dragonavance está en 0. En ese momento genero un número aleatorio del 1 al 8 usando `rand() % 8 + 1`, este número representa la cara ganadora que va a mostrar el dado al final. También calculo una dirección aleatoria hacia donde va a rodar el dado cuando caiga, para eso genero un ángulo aleatorio entre 0 y 360 grados y lo convierto a radianes. Luego uso las funciones trigonométricas `cos()` y `sin()` para obtener las componentes X y Z del vector de dirección, multiplicándolas por 3.0 para que el dado ruede esa distancia el 3 nos representa el radio total sobre el cual se va a mover.

```

if (dragonavance > -12.0f) {
    dragonavance -= avanceoffset * deltaTime;
    // Girar mientras cae
    dadoRotX += 300.0f * deltaTime;
    dadoRotY += 250.0f * deltaTime;
    dadoRotZ += 200.0f * deltaTime;
}

```

Mientras el dado no haya llegado al piso, es decir, mientras dragonavance sea mayor a -12.0, voy disminuyendo esta variable usando avanceoffset multiplicado por deltaTime, lo que hace que el dado caiga. Durante la caída, también hago que el dado gire rápidamente en los tres ejes sumándole a dadoRotX, dadoRotY y dadoRotZ valores de 300, 250 y 200 grados por segundo respectivamente, todo multiplicado por deltaTime para que sea independiente de la velocidad de los frames.

```

else {
    // Acercar poco a poco a donde va a quedar
    dadoLadoX *= 0.97f;
    dadoLadoZ *= 0.97f;

    dadoRotX += 50.0f * deltaTime;
    dadoRotY += 40.0f * deltaTime;
    dadoRotZ += 30.0f * deltaTime;

    //ajustar a la cara
    if (fabs(dadoLadoX) < 0.1f && fabs(dadoLadoZ) < 0.1f) {
        // 1 corazon
        if (numeroSalio == 1) { dadoRotX = -135.0f; dadoRotY = 45.0f; }
        // 2 rombos rojos
        if (numeroSalio == 2) { dadoRotX = 45.0f; dadoRotY = 45.0f; }
        // 3 corazones
        if (numeroSalio == 3) { dadoRotX = 45.0f; dadoRotY = -45.0f; }
        //4 rombos azules
        if (numeroSalio == 4) { dadoRotX = -135.0f; dadoRotY = -45.0f; }
        // 5 naipes
        if (numeroSalio == 5) { dadoRotX = 135.0f; dadoRotY = 45.0f; }
        //6 naipes verdes
        if (numeroSalio == 6) { dadoRotX = -45.0f; dadoRotY = 45.0f; }
        //7 triángulos
    }
}

```

Cuando el dado llega al piso, es decir cuando dragonavance es menor o igual a -12.0, entro en la fase de rodado. Aquí aplico un pequeño movimiento multiplicando dadoLadoX y dadoLadoZ por 0.97, esto hace que cada frame la velocidad lateral disminuya, simulando que el dado se va frenando poco a poco. También reduzco la velocidad de rotación a solo 50, 40 y 30 grados por segundo, mucho más lento que cuando caía.

Verifico si el dado ya se detuvo comparando si dadoLadoX y dadoLadoZ son menores a 0.1 en valor absoluto usando fabs(). Cuando esto sucede, ajusto la rotación final del dado según el número que salió. Tengo 8 casos, uno para cada cara del dado, donde asigno valores específicos de dadoRotX y dadoRotY para que la cara ganadora quede mirando hacia arriba.

```
//ajustar a la cara
if (fabs(dadoLadoX) < 0.1f && fabs(dadoLadoZ) < 0.1f) {
    // 1 corazon
    if (numeroSalio == 1) { dadoRotX = -135.0f; dadoRotY = 45.0f; }
    // 2 rombos rojos
    if (numeroSalio == 2) { dadoRotX = 45.0f; dadoRotY = 45.0f; }
    // 3 corazones
    if (numeroSalio == 3) { dadoRotX = 45.0f; dadoRotY = -45.0f; }
    //4 rombos azules
    if (numeroSalio == 4) { dadoRotX = -135.0f; dadoRotY = -45.0f; }
    // 5 naipes
    if (numeroSalio == 5) { dadoRotX = 135.0f; dadoRotY = 45.0f; }
    //6 naipes verdes
    if (numeroSalio == 6) { dadoRotX = -45.0f; dadoRotY = 45.0f; }
    //7 treboles
    if (numeroSalio == 7) { dadoRotX = -45.0f; dadoRotY = -45.0f; }
    //9 coraonez rojes
    if (numeroSalio == 8) { dadoRotX = 135.0f; dadoRotY = -45.0f; }
    dadoRotZ = 0.0f;
}
```

Finalmente, cuando el usuario suelta la tecla T, entro en el bloque else donde reseteo todas las variables a 0: dragonavance, dadoLadoX, dadoLadoZ, y todas las rotaciones. Esto deja el dado listo para hacer otra tirada y lo regresa a su estado inicial.

```
}
else {
    dragonavance = 0.0f;
    dadoLadoX = 0.0f;
    dadoLadoZ = 0.0f;
    dadoRotX = 0.0f;
    dadoRotY = 0.0f;
    dadoRotZ = 0.0f;
}
```

Ahora lo mas complicado fue el calcular la cara exacta en este caso tome en cuenta que nuestro octaedro pues tiene las caras a 45 grados ósea esta inclinada

de esa manera, entonces para rotarlas y ponerlas hacia arriba solo necesitamos rotaciones de $\pm 45^\circ$ o $\pm 135^\circ$

```
//Dado
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-1.5f + dadoLadoX, 10.5f + dragonavance, 4.0f + dadoLadoZ));
model = glm::rotate(model, dadoRotX * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, dadoRotY * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, dadoRotZ * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
dadoTexture.UseTexture();
meshList[8]->RenderMesh();
```

Ya en la instancia del dado lo que hacemos sumarle las variables de dadoLado y su componente y agregamos los rotates para los giros del dado.

Ejecución



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.

No hubo problemas durante la realización de los ejercicios.

3.- Conclusión:

La practica no fue complicada como tal, solamente estar tanteando en donde acomodar las cosas y separar las rejas en blender fue el problema y lo más tardado diría yo, el ejercicio del dado presento una dificultad mayor sobre todo para la parte de hacer que cayera rodando y que nos diera una cara exacta, me parece que aprendí un poco más de lo básico de animaciones, es un buen ejercicio sobre todo el de las puertas.